

前端性能优化面试题-30题

1. 什么是前端性能优化？为什么重要？

参考答案：

前端性能优化是指通过各种技术手段和策略，提升网页加载速度、渲染效率和用户交互体验的过程。

重要性：

- **用户体验：**页面加载速度直接影响用户满意度和留存率
- **业务价值：**研究表明页面加载时间每增加1秒，转化率下降7%
- **SEO影响：**搜索引擎将页面速度作为排名因素之一
- **资源成本：**减少带宽消耗和服务器压力

2. 前端性能优化的核心指标有哪些？

参考答案：

Core Web Vitals（核心网页指标）：

- **LCP（Largest Contentful Paint）：**最大内容绘制时间，理想值<2.5s
- **FID（First Input Delay）：**首次输入延迟，理想值<100ms
- **CLS（Cumulative Layout Shift）：**累积布局偏移，理想值<0.1

其他重要指标：

- **FCP（First Contentful Paint）：**首次内容绘制
- **TTI（Time to Interactive）：**可交互时间
- **FMP（First Meaningful Paint）：**首次有意义绘制

3. 如何减少HTTP请求数量？

参考答案：

- **资源合并**：CSS/JS文件合并，雪碧图（CSS Sprites）
- **内联资源**：小图片转base64，关键CSS内联
- **懒加载**：图片、组件按需加载
- **缓存策略**：利用浏览器缓存减少重复请求
- **CDN加速**：静态资源使用CDN分发
- **HTTP/2**：利用多路复用特性

代码块

```
1 // 图片懒加载示例
2 const lazyImages = document.querySelectorAll('img[data-src]');
3 const imageObserver = new IntersectionObserver((entries) => {
4   entries.forEach(entry => {
5     if (entry.isIntersecting) {
6       const img = entry.target;
7       img.src = img.dataset.src;
8       imageObserver.unobserve(img);
9     }
10   });
11 });
12 lazyImages.forEach(img => imageObserver.observe(img));
```

4. 什么是关键渲染路径？如何优化？

参考答案：

关键渲染路径是浏览器将HTML、CSS和JavaScript转换为屏幕像素的步骤序列。

优化策略：

- **减少关键资源数量**：内联关键CSS，延迟非关键资源
- **减少关键字节数**：压缩资源，移除未使用代码
- **缩短关键路径长度**：优化资源加载顺序

```

1 关键CSS内联 -->
2  <style>
3    /* 首屏关键样式 */
4    .header { display: flex; }
5  </style>
6
7  <!-- 非关键CSS异步加载 -->
8  <link rel="preload" href="styles.css" as="style"
    onload="this.onload=null;this.rel='stylesheet'">

```

5. CSS性能优化有哪些方法？

参考答案：

- **选择器优化**：避免复杂选择器，减少嵌套层级
- **CSS压缩**：移除空格、注释，合并相同规则
- **避免@import**：使用link标签代替@import
- **CSS3硬件加速**：使用transform、opacity触发GPU加速
- **移除未使用CSS**：工具如PurgeCSS清理无用样式

代码块

```

1  /* 避免复杂选择器 */
2  /* 不好 */
3  .nav ul li a span { color: red; }
4
5  /* 好 */
6  .nav-link-text { color: red; }
7
8  /* 触发硬件加速 */
9  .animated {
10    transform: translateZ(0);
11    will-change: transform;
12  }

```

6. JavaScript性能优化策略有哪些？

参考答案：

- **代码分割**：按需加载，动态import
- **Tree Shaking**：移除未使用代码
- **压缩混淆**：减小文件体积
- **Web Workers**：将计算密集任务移到后台线程
- **避免阻塞**：使用async/defer属性

代码块

```
1  // 动态导入
2  const loadModule = async () => {
3      const module = await import('./heavy-module.js');
4      module.init();
5  };
6
7  // Web Workers
8  const worker = new Worker('calculation.js');
9  worker.postMessage(data);
10 worker.onmessage = (e) => {
11     console.log('Result:', e.data);
12 };
```

7. 图片优化有哪些技术？

参考答案：

- **格式选择**：WebP > JPEG > PNG，根据场景选择
- **尺寸优化**：响应式图片，srcset属性
- **压缩**：有损/无损压缩，工具如TinyPNG
- **懒加载**：viewport外图片延迟加载
- **雪碧图**：小图标合并减少请求

代码块

```
1 <!-- 响应式图片 -->
2 <picture>
3   <source media="(min-width: 800px)" srcset="large.webp" type="image/webp">
4   <source media="(min-width: 400px)" srcset="medium.webp" type="image/webp">
5   
6 </picture>
```

8. 什么是浏览器缓存？如何配置？

参考答案：

浏览器缓存是将资源存储在本地，减少网络请求的机制。

缓存类型：

- 强缓存：Cache-Control, Expires
- 协商缓存：ETag, Last-Modified

配置策略：

代码块

```
1 # 静态资源长期缓存
2 Cache-Control: max-age=31536000, immutable
3
4 # HTML文件不缓存
5 Cache-Control: no-cache
6
7 # API接口短期缓存
8 Cache-Control: max-age=300
```

9. CDN的工作原理和优势是什么？

参考答案：

CDN（内容分发网络）将内容缓存到全球各地的边缘服务器，用户从最近的服务器获取资源。

优势：

- **减少延迟**：就近访问，降低网络延迟
- **减轻源站压力**：分散请求到边缘节点
- **提高可用性**：多节点冗余，提升稳定性
- **节省带宽**：减少源站带宽消耗

使用场景：

- 静态资源（CSS、JS、图片）
- 视频、音频等大文件
- API接口加速

10. 什么是预加载？有哪些类型？

参考答案：

预加载是提前获取用户可能需要的资源，提升后续访问速度。

类型：

- **DNS预解析**： `<link rel="dns-prefetch" href="//example.com">`
- **预连接**： `<link rel="preconnect" href="//fonts.googleapis.com">`
- **资源预加载**： `<link rel="preload" href="style.css" as="style">`
- **页面预取**： `<link rel="prefetch" href="/next-page.html">`

代码块

```
1  <!-- 预加载关键字体 -->
2  <link rel="preload" href="/fonts/main.woff2" as="font" type="font/woff2"
   crossorigin>
3
4  <!-- 预取下一页 -->
5  <link rel="prefetch" href="/product-detail.html">
```

11. 如何优化首屏加载时间？

参考答案：

- **关键资源优先**：内联关键CSS，优先加载首屏内容
- **代码分割**：只加载首屏必需代码
- **服务端渲染（SSR）**：减少客户端渲染时间
- **骨架屏**：提供视觉反馈，改善感知性能
- **资源预加载**：提前加载关键资源

代码块

```
1  // 代码分割示例
2  const HomePage = lazy(() => import('./HomePage'));
3  const ProductPage = lazy(() => import('./ProductPage'));
4
5  function App() {
6    return (
7      <Suspense fallback={<SkeletonLoader />}>
8        <Routes>
9          <Route path="/" element={<HomePage />} />
10         <Route path="/product" element={<ProductPage />} />
11        </Routes>
12      </Suspense>
13    );
14  }
```

12. 什么是虚拟滚动？如何实现？

参考答案：

虚拟滚动是只渲染可视区域内的列表项，大幅提升长列表性能的技术。

实现原理：

- 计算可视区域能显示的项目数量

- 根据滚动位置计算当前应渲染的项目
- 动态创建/销毁DOM元素

代码块

```
1  class VirtualList {
2    constructor(container, itemHeight, totalItems) {
3      this.container = container;
4      this.itemHeight = itemHeight;
5      this.totalItems = totalItems;
6      this.visibleCount = Math.ceil(container.clientHeight / itemHeight);
7      this.startIndex = 0;
8
9      this.init();
10   }
11
12   init() {
13     this.container.style.height = this.totalItems * this.itemHeight + 'px';
14     this.container.addEventListener('scroll', this.onScroll.bind(this));
15     this.render();
16   }
17
18   onScroll() {
19     this.startIndex = Math.floor(this.container.scrollTop / this.itemHeight);
20     this.render();
21   }
22
23   render() {
24     const endIndex = Math.min(this.startIndex + this.visibleCount,
25                               this.totalItems);
26     // 渲染 startIndex 到 endIndex 的项目
27   }
```

13. 如何进行Bundle分析和优化?

参考答案:

分析工具:

- **webpack-bundle-analyzer**: 可视化bundle组成

- **source-map-explorer**: 分析源码占用
- **bundlephobia**: 分析npm包大小

优化策略:

- **代码分割**: 按路由/功能分割
- **Tree Shaking**: 移除未使用代码
- **外部依赖**: 大型库使用CDN
- **动态导入**: 按需加载模块

代码块

```
1  // webpack配置示例
2  module.exports = {
3    optimization: {
4      splitChunks: {
5        chunks: 'all',
6        cacheGroups: {
7          vendor: {
8            test: /[\\/]node_modules[\\/]/,
9            name: 'vendors',
10           chunks: 'all',
11         },
12       },
13     },
14   },
15   };
```

14. 什么是Service Worker? 如何用于性能优化?

参考答案:

Service Worker是运行在后台的脚本, 可以拦截网络请求, 实现离线缓存和推送通知。

性能优化应用:

- **缓存策略**: 实现复杂的缓存逻辑
- **离线访问**: 缓存关键资源, 支持离线浏览

- **预缓存**：在空闲时预加载资源
- **网络优化**：智能选择缓存或网络

代码块

```
1  // service-worker.js
2  self.addEventListener('fetch', event => {
3    if (event.request.destination === 'image') {
4      event.respondWith(
5        caches.match(event.request).then(response => {
6          return response || fetch(event.request).then(fetchResponse => {
7            const responseClone = fetchResponse.clone();
8            caches.open('images').then(cache => {
9              cache.put(event.request, responseClone);
10             });
11            return fetchResponse;
12          });
13        })
14      );
15    }
16  });
```

15. 如何优化移动端性能？

参考答案：

- **触摸优化**：使用touch事件，避免300ms延迟
- **视口配置**：正确设置viewport meta标签
- **图片适配**：使用合适尺寸和格式
- **网络优化**：考虑弱网环境，实现降级策略
- **电池优化**：减少CPU密集操作

代码块

```
1  <!-- 移动端视口配置 -->
2  <meta name="viewport" content="width=device-width, initial-scale=1.0, user-
3    scalable=no">
```

```
4 <!-- 避免300ms延迟 -->
5 <meta name="viewport" content="width=device-width, initial-scale=1.0, touch-
  action=manipulation">
```

代码块

```
1 /* 移动端优化 */
2 .touch-element {
3   touch-action: manipulation; /* 避免双击缩放延迟 */
4   -webkit-tap-highlight-color: transparent; /* 移除点击高亮 */
5 }
```

16. 什么是关键CSS？如何提取和使用？

参考答案：

关键CSS是渲染首屏内容所必需的最小CSS集合。

提取方法：

- **工具：** Critical、Penthouse、UnCSS
- **手动分析：** 识别首屏元素对应样式
- **自动化：** 构建流程中自动提取

使用策略：

代码块

```
1 <!-- 内联关键CSS -->
2 <style>
3   /* 首屏关键样式 */
4   .header { display: flex; height: 60px; }
5   .hero { min-height: 400px; }
6 </style>
7
8 <!-- 异步加载完整CSS -->
9 <link rel="preload" href="/styles/main.css" as="style"
  onload="this.onload=null;this.rel='stylesheet'">
```

```
10 <noscript><link rel="stylesheet" href="/styles/main.css"></noscript>
```

17. 如何实现资源的预加载和懒加载？

参考答案：

预加载（Preloading）：

代码块

```
1 <!-- 预加载关键资源 -->
2 <link rel="preload" href="/fonts/main.woff2" as="font" crossorigin>
3 <link rel="preload" href="/images/hero.jpg" as="image">
4
5 <!-- JavaScript预加载 -->
6 <script>
7   const link = document.createElement('link');
8   link.rel = 'preload';
9   link.href = '/api/data.json';
10  link.as = 'fetch';
11  document.head.appendChild(link);
12 </script>
```

懒加载（Lazy Loading）：

代码块

```
1 // 图片懒加载
2 const imageObserver = new IntersectionObserver((entries) => {
3   entries.forEach(entry => {
4     if (entry.isIntersecting) {
5       const img = entry.target;
6       img.src = img.dataset.src;
7       img.classList.remove('lazy');
8       imageObserver.unobserve(img);
9     }
10  });
11 });
12
13 // 组件懒加载
```

```
14  const LazyComponent = React.lazy(() => import('./HeavyComponent'));
```

18. 什么是Web Vitals? 如何监控和优化?

参考答案:

Web Vitals是Google提出的用户体验质量指标集合。

核心指标:

- **LCP**: 最大内容绘制 (<2.5s)
- **FID**: 首次输入延迟 (<100ms)
- **CLS**: 累积布局偏移 (<0.1)

监控方法:

代码块

```
1  // 使用web-vitals库
2  import {getCLS, getFID, getFCP, getLCP, getTTFB} from 'web-vitals';
3
4  getCLS(console.log);
5  getFID(console.log);
6  getFCP(console.log);
7  getLCP(console.log);
8  getTTFB(console.log);
9
10 // 自定义上报
11 function sendToAnalytics(metric) {
12   fetch('/analytics', {
13     method: 'POST',
14     body: JSON.stringify(metric)
15   });
16 }
```

19. 如何优化JavaScript执行性能?

参考答案：

- **减少主线程阻塞**：使用requestIdleCallback
- **优化算法复杂度**：选择高效算法和数据结构
- **避免内存泄漏**：及时清理事件监听器和定时器
- **使用Web Workers**：将计算密集任务移到后台
- **代码分割**：按需加载，减少初始bundle大小

代码块

```
1  // 时间切片优化长任务
2  function processLargeArray(array, callback) {
3      const chunk = 1000;
4      let index = 0;
5
6      function processChunk() {
7          const start = performance.now();
8
9          while (index < array.length && (performance.now() - start) < 5) {
10             // 处理数组项
11             processItem(array[index++]);
12         }
13
14         if (index < array.length) {
15             requestIdleCallback(processChunk);
16         } else {
17             callback();
18         }
19     }
20
21     processChunk();
22 }
```

20. 什么是HTTP/2？对性能有什么影响？

参考答案：

HTTP/2是HTTP协议的第二个主要版本，带来显著性能提升。

主要特性：

- **多路复用**：单连接并行处理多个请求
- **头部压缩**：HPACK算法压缩HTTP头
- **服务器推送**：主动推送资源给客户端
- **二进制分帧**：更高效的数据传输

性能影响：

- 减少连接数，降低延迟
- 消除队头阻塞问题
- 减少网络开销
- 更好的带宽利用率

代码块

```
1 // HTTP/2服务器推送示例 (Node.js)
2 const http2 = require('http2');
3 const server = http2.createSecureServer(options);
4
5 server.on('stream', (stream, headers) => {
6   if (headers[':path'] === '/') {
7     // 推送CSS和JS资源
8     stream.pushStream({':path': '/styles.css'}, (err, pushStream) => {
9       pushStream.respondWithFile('./styles.css');
10    });
11   }
12 });
```

21. 如何进行前端性能监控？

参考答案：

监控指标：

- **加载性能**：FCP, LCP, TTI
- **运行时性能**：FPS, 内存使用, CPU占用
- **用户体验**：FID, CLS, 错误率

监控方案：

代码块

```
1  // Performance Observer API
2  const observer = new PerformanceObserver((list) => {
3    list.getEntries().forEach((entry) => {
4      if (entry.entryType === 'largest-contentful-paint') {
5        console.log('LCP:', entry.startTime);
6        // 上报数据
7        sendMetric('LCP', entry.startTime);
8      }
9    });
10 });
11
12 observer.observe({entryTypes: ['largest-contentful-paint']});
13
14 // 自定义性能监控
15 class PerformanceMonitor {
16   static trackPageLoad() {
17     window.addEventListener('load', () => {
18       const perfData = performance.timing;
19       const metrics = {
20         dns: perfData.domainLookupEnd - perfData.domainLookupStart,
21         tcp: perfData.connectEnd - perfData.connectStart,
22         ttfb: perfData.responseStart - perfData.navigationStart,
23         domReady: perfData.domContentLoadedEventEnd - perfData.navigationStart
24       };
25       this.sendMetrics(metrics);
26     });
27   }
28 }
```

22. 什么是Tree Shaking? 如何配置?

参考答案：

Tree Shaking是移除JavaScript中未使用代码的优化技术，基于ES6模块的静态结构。

工作原理：

- 分析模块依赖关系
- 标记使用的代码
- 移除未使用的代码

配置示例：

代码块

```
1  // webpack.config.js
2  module.exports = {
3    mode: 'production',
4    optimization: {
5      usedExports: true,
6      sideEffects: false, // 标记包为无副作用
7    },
8  };
9
10 // package.json
11 {
12   "sideEffects": [
13     "*.css",
14     "*.scss",
15     "./src/polyfills.js"
16   ]
17 }
18
19 // 正确的导入方式
20 import { debounce } from 'lodash-es'; // 支持tree shaking
21 // 避免
22 import _ from 'lodash'; // 导入整个库
```

23. 如何优化CSS动画性能？

参考答案：

- 使用transform和opacity：触发GPU加速，避免重排重绘
- will-change属性：提示浏览器优化动画元素
- 避免动画布局属性：width、height、margin等
- 使用CSS3动画：优于JavaScript动画

- 合理使用硬件加速：避免过度使用导致内存问题

代码块

```
1  /* 高性能动画 */
2  .optimized-animation {
3      will-change: transform;
4      transform: translateZ(0); /* 创建合成层 */
5      transition: transform 0.3s ease-out;
6  }
7
8  .optimized-animation:hover {
9      transform: translateX(100px) scale(1.1);
10 }
11
12 /* 避免的动画属性 */
13 .bad-animation {
14     transition: width 0.3s; /* 会触发重排 */
15 }
16
17 /* 使用transform替代 */
18 .good-animation {
19     transform: scaleX(1.2); /* 只触发合成 */
20 }
```

24. 什么是资源提示（Resource Hints）？

参考答案：

资源提示是HTML5规范，允许开发者向浏览器提供关于资源加载的提示。

类型：

- dns-prefetch：DNS预解析
- preconnect：预连接
- preload：预加载
- prefetch：预取
- prerender：预渲染

代码块

```
1  <!-- DNS预解析 -->
2  <link rel="dns-prefetch" href="//fonts.googleapis.com">
3
4  <!-- 预连接（包含DNS解析、TCP握手、TLS协商） -->
5  <link rel="preconnect" href="//fonts.gstatic.com" crossorigin>
6
7  <!-- 预加载当前页面需要的资源 -->
8  <link rel="preload" href="/critical.css" as="style">
9  <link rel="preload" href="/hero.jpg" as="image">
10
11 <!-- 预取用户可能访问的资源 -->
12 <link rel="prefetch" href="/next-page.html">
13
14 <!-- 预渲染整个页面 -->
15 <link rel="prerender" href="/landing-page.html">
```

25. 如何实现代码分割（Code Splitting）？

参考答案：

代码分割是将代码拆分成多个bundle，实现按需加载的技术。

分割策略：

- 入口分割：多个入口点
- 动态导入：import()语法
- 第三方库分割：vendor chunk

代码块

```
1  // 动态导入
2  const loadComponent = async () => {
3    const { default: Component } = await import('./HeavyComponent');
4    return Component;
5  };
6
```

```
7 // React代码分割
8 const LazyComponent = React.lazy(() => import('./LazyComponent'));
9
10 function App() {
11   return (
12     <Suspense fallback={<div>Loading...</div>}>
13       <LazyComponent />
14     </Suspense>
15   );
16 }
17
18 // webpack配置
19 module.exports = {
20   optimization: {
21     splitChunks: {
22       chunks: 'all',
23       cacheGroups: {
24         vendor: {
25           test: /[\\/]node_modules[\\/]/,
26           name: 'vendors',
27           chunks: 'all',
28         },
29         common: {
30           name: 'common',
31           minChunks: 2,
32           chunks: 'all',
33         }
34       }
35     }
36   }
37 };
```

26. 什么是关键渲染路径优化?

参考答案:

关键渲染路径优化是指优化浏览器渲染页面的关键步骤，减少首屏渲染时间。

优化步骤:

1. 分析关键资源：识别渲染首屏必需的资源
2. 减少关键资源数量：合并、内联关键资源

3. 压缩关键字数：压缩CSS、JS、HTML

4. 优化加载顺序：优先加载关键资源

代码块

```
1  <!-- 优化示例 -->
2  <!DOCTYPE html>
3  <html>
4  <head>
5      <!-- 内联关键CSS -->
6      <style>
7          /* 首屏关键样式 */
8          body { margin: 0; font-family: Arial; }
9          .header { height: 60px; background: #333; }
10     </style>
11
12     <!-- 预加载字体 -->
13     <link rel="preload" href="/fonts/main.woff2" as="font" crossorigin>
14 </head>
15 <body>
16     <!-- 首屏内容 -->
17     <header class="header">...</header>
18
19     <!-- 异步加载非关键CSS -->
20     <link rel="preload" href="/styles/main.css" as="style"
21         onload="this.onload=null;this.rel='stylesheet'">
22
23     <!-- 延迟加载非关键JS -->
24     <script src="/js/main.js" defer></script>
25 </body>
26 </html>
```

27. 如何优化长列表渲染性能？

参考答案：

优化策略：

- 虚拟滚动：只渲染可视区域
- 分页加载：按需加载数据

- **防抖节流**：优化滚动事件
- **使用key优化**：React中正确使用key
- **避免内联函数**：减少不必要的重渲染

代码块

```
1  // 虚拟滚动实现
2  class VirtualScroller {
3    constructor(container, itemHeight, items) {
4      this.container = container;
5      this.itemHeight = itemHeight;
6      this.items = items;
7      this.visibleCount = Math.ceil(container.clientHeight / itemHeight);
8      this.startIndex = 0;
9
10     this.init();
11   }
12
13   init() {
14     this.container.addEventListener('scroll',
15     this.throttle(this.onScroll.bind(this), 16));
16     this.render();
17   }
18
19   onScroll() {
20     const scrollTop = this.container.scrollTop;
21     this.startIndex = Math.floor(scrollTop / this.itemHeight);
22     this.render();
23   }
24
25   render() {
26     const endIndex = Math.min(this.startIndex + this.visibleCount + 1,
27     this.items.length);
28     const visibleItems = this.items.slice(this.startIndex, endIndex);
29
30     // 渲染可见项目
31     this.container.innerHTML = visibleItems.map((item, index) =>
32     `<div style="height: ${this.itemHeight}px; transform:
33     translateY(${(this.startIndex + index) * this.itemHeight}px)">
34       ${item.content}
35     </div>`
36     ).join('');
37   }
38
39   throttle(func, delay) {
```

```

37     let timer = null;
38     return function() {
39         if (!timer) {
40             timer = setTimeout(() => {
41                 func.apply(this, arguments);
42                 timer = null;
43             }, delay);
44         }
45     };
46 }
47 }

```

28. 什么是PWA？如何提升性能？

参考答案：

PWA（Progressive Web App）是使用现代Web技术构建的应用，提供类似原生应用的体验。

性能优化特性：

- **Service Worker**：离线缓存和后台同步
- **App Shell模式**：快速加载应用外壳
- **预缓存策略**：关键资源预缓存
- **推送通知**：提升用户参与度

代码块

```

1  // service-worker.js
2  const CACHE_NAME = 'app-v1';
3  const urlsToCache = [
4      '/',
5      '/styles/main.css',
6      '/scripts/main.js',
7      '/images/icon.png'
8  ];
9
10 // 安装时预缓存资源
11 self.addEventListener('install', event => {
12     event.waitUntil(
13         caches.open(CACHE_NAME)

```

```

14     .then(cache => cache.addAll(urlsToCache))
15   );
16 });
17
18 // 拦截请求, 优先从缓存读取
19 self.addEventListener('fetch', event => {
20   event.respondWith(
21     caches.match(event.request)
22       .then(response => {
23         return response || fetch(event.request);
24       })
25   );
26 });
27
28 // 注册Service Worker
29 if ('serviceWorker' in navigator) {
30   navigator.serviceWorker.register('/sw.js');
31 }

```

29. 如何进行前端性能测试?

参考答案:

测试工具:

- **Lighthouse**: 综合性能评估
- **WebPageTest**: 详细性能分析
- **Chrome DevTools**: 实时性能监控
- **GTmetrix**: 页面速度测试

测试策略:

代码块

```

1 // 自动化性能测试
2 const lighthouse = require('lighthouse');
3 const chromeLauncher = require('chrome-launcher');
4
5 async function runLighthouse(url) {
6   const chrome = await chromeLauncher.launch({chromeFlags: ['--headless']});
7   const options = {logLevel: 'info', output: 'html', port: chrome.port};

```



```

8     const runnerResult = await lighthouse(url, options);
9
10    const score = runnerResult.report;
11    console.log('Performance score:',
runnerResult.lhr.categories.performance.score * 100);
12
13    await chrome.kill();
14  }
15
16  // 性能预算设置
17  const performanceBudget = {
18    'first-contentful-paint': 2000,
19    'largest-contentful-paint': 2500,
20    'cumulative-layout-shift': 0.1,
21    'total-blocking-time': 300
22  };
23
24  // 监控关键指标
25  function monitorPerformance() {
26    new PerformanceObserver((list) => {
27      list.getEntries().forEach((entry) => {
28        const metric = entry.name || entry.entryType;
29        const value = entry.startTime || entry.value;
30
31        if (performanceBudget[metric] && value > performanceBudget[metric]) {
32          console.warn(`Performance budget exceeded: ${metric} = ${value}ms`);
33        }
34      });
35    }).observe({entryTypes: ['paint', 'largest-contentful-paint', 'layout-
shift']}));
36  }

```

30. 前端性能优化的最佳实践总结？

参考答案：

加载优化：

- 减少HTTP请求数量
- 启用Gzip/Brotli压缩
- 使用CDN加速
- 实施缓存策略

- 优化关键渲染路径

运行时优化：

- 避免长任务阻塞主线程
- 使用虚拟滚动处理长列表
- 合理使用Web Workers
- 优化动画性能
- 防止内存泄漏

资源优化：

- 图片格式和尺寸优化
- 代码分割和懒加载
- Tree Shaking移除无用代码
- 字体加载优化

监控和测试：

- 建立性能监控体系
- 设置性能预算
- 定期进行性能测试
- 关注Core Web Vitals指标

开发流程：

- 性能优先的开发理念
- 自动化构建优化
- 持续性能监控
- 团队性能意识培养